Documentation of GUI-based NLSA for Analyzing SWAXS Data

Computational Biophysics Group, Physics Department,
University of Wisconsin-Milwaukee, Milwaukee, WI 53211, USA
Copyright © 2025 UWM Computational Biophysics Group
Contact information: Ahmad Hosseinizadeh (hosseina@uwm.edu)

1. Overview

This document provides a comprehensive explanation of the GUI code for analyzing SWAXS data described in the main article. It includes detailed descriptions of the user interface components, their functions, and sample screenshots.

2. GUI Code Location

The program is named 'gui_nlsa.py' and it is stored inside the */gui* folder.

3. Code Workflow

This program provides a GUI-driven data analysis tool that:

- Loads a dataset,
- Prepares it for processing,
- Executes an MPI-based computation using *multi_nN_prepare_dsq.py* (for preparing the squared distance file),
- Performs sigma optimization using Ferguson analysis,
- Applies diffusion map dimensionality reduction,
- Calculates and visualizes correlation matrices based on eigenfunctions.

The program's logic is incorporated into three important functions listed as:

a) run_analysis()

b) click_run()

c) run_nlsa()

Below, we describe these functions.

a) run_analysis(self)

Initializes and coordinates the full analysis pipeline, ensuring UI feedback and error handling.

- **Steps:**

1. File Validation
    - Ensures a file has been selected and exists on disk.
2. UI Initialization
    - Updates the GUI status bar and shows the progress bar to the user.
3. Clean-up of Previous Selections
    - Removes any previous selection graphics from the interface.
4. Save Data
    - Saves current input or state to disk.
5. Trigger Main Analysis
    - Calls *click_run()* to perform the computational heavy lifting.
6. Update Visualization
    - Displays results in the form of a heatmap on the GUI.
7. Error Handling
    - Catches and reports errors via a message box.

b) click_run(self)

Handles the core data processing and scientific computation logic.

- **Steps:**

1. Setting Up Input Data

    - Writes the input dataset to an HDF5 file so it can be read by external tools.

2. External MPI-based Computation

- Executes the external Python script using *mpiexec* for parallel processing.
- Computes pairwise squared distances with the necessary input parameters.

3. Live Output Handling

- Streams the output of the external script to the console.
- Updates the progress bar incrementally up to 65%.

4. Ferguson Sigma Optimization

- Runs Ferguson's method to compute the optimal sigma value for each neighborhood size.
- Saves results in sigma_opt.h5.
- Updates progress bar to ~75%.

5. Diffusion Map Embedding

- For each (nN, sigma_factor) pair:
  - Computes eigenfunctions and eigenvalues.
  - Stores them in nested lists (eigVec_list, eigVal_list).
- Progress bar updated to ~85%.

6. Saving Eigen Decomposition Results

- Saves computed eigenfunctions and eigenvalues to HDF5 files.

7. Computing the Eigenfunctions Correlation Matrix

- Compares each pair of eigenfunctions.
- Counts how many components are strongly aligned using a threshold (0.90).
- Filters non-noisy dimensions using *self.corr_no_noisy()*.

8. Heatmap Matrix Rearrangement

- Reshapes and rearranges the correlation matrix:

- *eigVec_corr*: Raw correlation,
- *eigVec_shuffled*: Restructured for visualization,
- *eigVec_heatmap*: Final lower + upper triangular format.

9. Final Step

- Sets progress bar to 100%.
- Returns the final heatmap to be displayed on the GUI.

- **Output Files:**

- *data_file_for_sna.h5*: Raw input data.
- *dSq_nN_*.h5*: Distance matrix files.
- *sigma_opt.h5*: Optimized sigma values.
- *eigVec_list.h5*: Eigenfunctions for each setting.
- *eigVal_list.h5*: Corresponding eigenvalues.
- *eigVec_heatmap.h5*: Final correlation/heatmap data.

c) **run_nlsa()**

The *run_nlsa* code is a core component in the pipeline, designed to perform Nonlinear Laplacian Spectral Analysis (NLSA). It integrates several data processing steps, system subprocess calls, file handling routines, and user interface updates. Below is a step-by-step explanation of the logic and function of this method.

- **Steps:**

1. Initialization and Parameter Extraction

- Clears previous selections and updates the progress bar.
- Reads neighborhood size (nN_1) and kernel width (sigma_1) from the user.
- Extracts the corresponding eigenfunction data from the *eigVec_list.h5* file and stores them in class variables.

2. *mu* and *psi* Computation

- Computes *mu* as the square of the first column of eigenfunctions.
- Computes *psi* as the remaining eigenfunctions normalized by *mu*.
- Saves these arrays to the *mu_psi.h5* file using the helper function *write_h5*.

3. Dot Product Computation via MPI

- Executes the Python script *run_sna_.py* using MPI to perform the dot product computations.
- Monitors the output in real-time and updates the progress bar accordingly.

4. Cleanup

- Calls *post_sna_cleanup_.py* to clean up intermediate files and data after the dot product step.
- If multiple concatenations are used (c > 1), an additional cleanup is triggered.

5. NLSA Computation

- Executes the NLSA algorithm via the *run_nlsa.py* script with relevant parameters and precomputed *mu_psi* values.
- Another cleanup pass is made afterward using square mode to tidy the NLSA output.

6. Visualization and Plotting

- Loads the resulting singular vectors V and singular values S from a file, named *usv_nlsa_N{N}_nN{nN}_c{C}_sigma{sigma}_nCopy{nCopy}.h5*.
- Plots chronos and saves them in the *chrono.png* file.
- Plots singular values and saves them in the *nlsa_sing_val.png* file.

7. GUI Visualization Embedding

- Creates subplots for each chrono within the GUI using matplotlib and embeds them in

the application.

- Adds interactive red boxes (hidden by default) for possible visual emphasis.
- Shows a plot of the singular values.

8. Final UI Updates

- Updates the progress bar to 100%.
- Adds a new tab to the GUI labeled with the selected nN and sigma values.
- Clears selected indices in preparation for future runs.

- **Output Files:**

1. *mu_psi.h5*

Contents:

- mu – weight vector derived from the firseigenfunction
- psi – normalized eigenfunctions excluding the first one

2. *xtx.h5*

   Placeholder name used in dot product computation (might not be created depending on the script's internal behavior)

3. *usv_nlsa_N{N}_nN{nN}_c{C}_sigma{sigma}_nCopy{nCopy}.h5*

   Contents:

   - U – NLSA topos (spatial modes)
   - S – NLSA singular values
   - V – NLSA chronos (temporal modes)
   -

4. *chrono.png*

   Line plots of chronos V from NLSA

5. *nlsa_sing_val.png*

   Line plot of singular values S from NLSA

The major computational steps and outputs of the code are:

A. Running the GUI code

1.Create a conda environment on the machine where you run the GUI.

2. Install the below-listed modules.

Numpy, scipy, h5py, matplotlib, mpi4py, mpich

3. Go to the */gui* folder where the NLSA script (gui_nlsa.py) is stored and run the code. This will show the GUI on the screen.

B. GUI Components Explanation

The startup GUI contains two sections:

- Settings

This part contains buttons, text boxes, drop-down, text labels, and a progress bar for selecting a data file, setting parameters, and showing the status of the parameter search and NLSA analysis (see Figure 1 below).

- Interactive Heatmap

In this section, a clickable heatmap is displayed, as shown below.

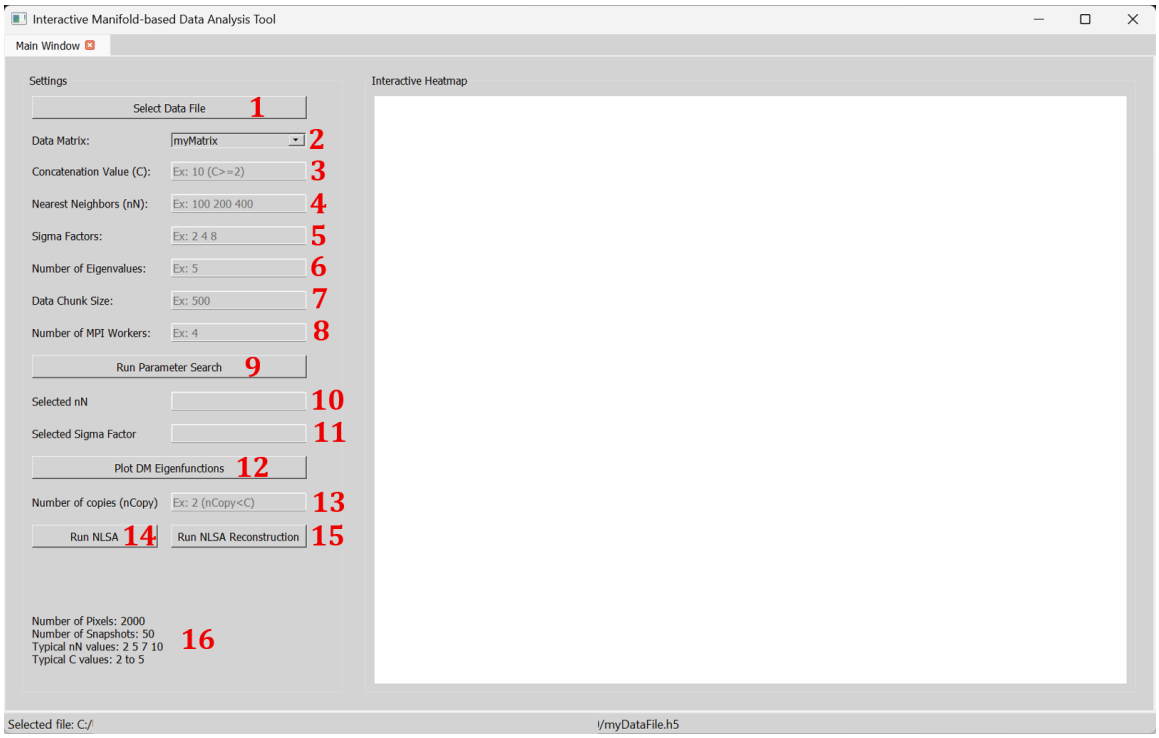

Figure 1. The startup window of the GUI

In the Settings area, there are these buttons:

1. Select Data File:

This is a button used for selecting the .h5 data file. Once we select the data file, it shows the Data Matrix drop-down at #2 and data info at#16

Note: It accepts only *.*h5* data files

2. Data Matrix

This is a dropdown list of the variable names inside the selected data

3. Concatenation Value:

This is a text box that accepts only one integer value. This concatenation value refers to 'c' in the backend program.

4. Nearest neighbors (NN):

This is a text box that accepts a list of space-separated integer values as a list of NN.

5. Sigma Factors:

A textbox accepting the list of space-separated integers as a list of sigma factors.

6. Number of Eigenvalues:

It is a textbox that accepts a single integer value specifying the number of eigenvalues for parameter search.

7. Data Chunk Size:

This field takes a single integer value. We can specify this value based on the data size and the number of MPI workers to divide the data equally among the workers for faster analysis.

8. Number of MPI Workers:

This is a text field that accepts a single integer value. This specifies the number of parallel workers we want to use for the given parameter analysis.

9. Run Parameter Search:

This button initiates the parameter search analysis. It shows the progress bar #16, soon after clicking it.

10. Selected nN

This box is automatically filled with the selected nN value (see Figure 2 below) that will be used for plotting eigenfunctions and completing the remainder of the NLSA pipeline.

11. Selected Sigma Factor:

Similar to the above step, this box is automatically filled with the selected sigma factor (Figure 2) that will be used for plotting eigenfunctions and completing the remainder of the NLSA pipeline.

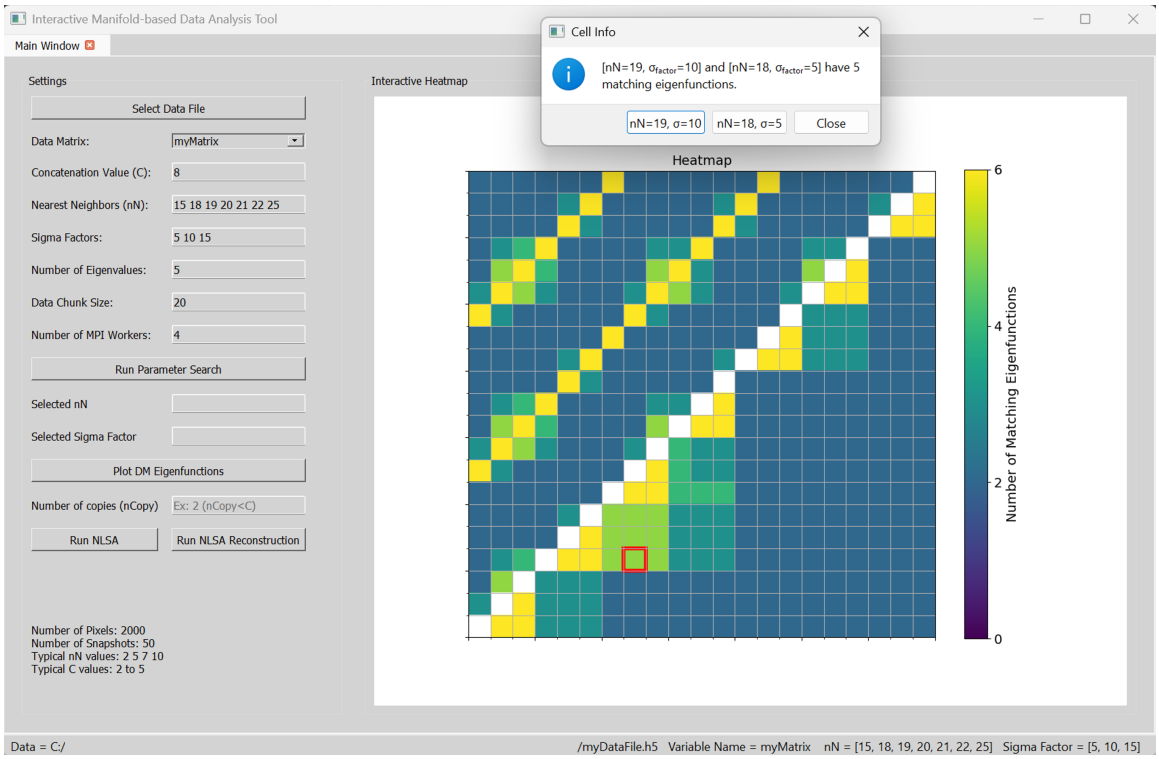

Figure 2. When we click on any heatmap cell, it will show the Cell info window as shown in the image below, and then we can select any combination of nN and sigma values to populate these fields for plotting eigenfunctions.

12. Plot DM Eigenfunctions:

This button is used to plot the diffusion map eigenfunctions for the selected nN and sigma values.These plots are displayed in the new tab, as demonstrated in Figure 3 below.

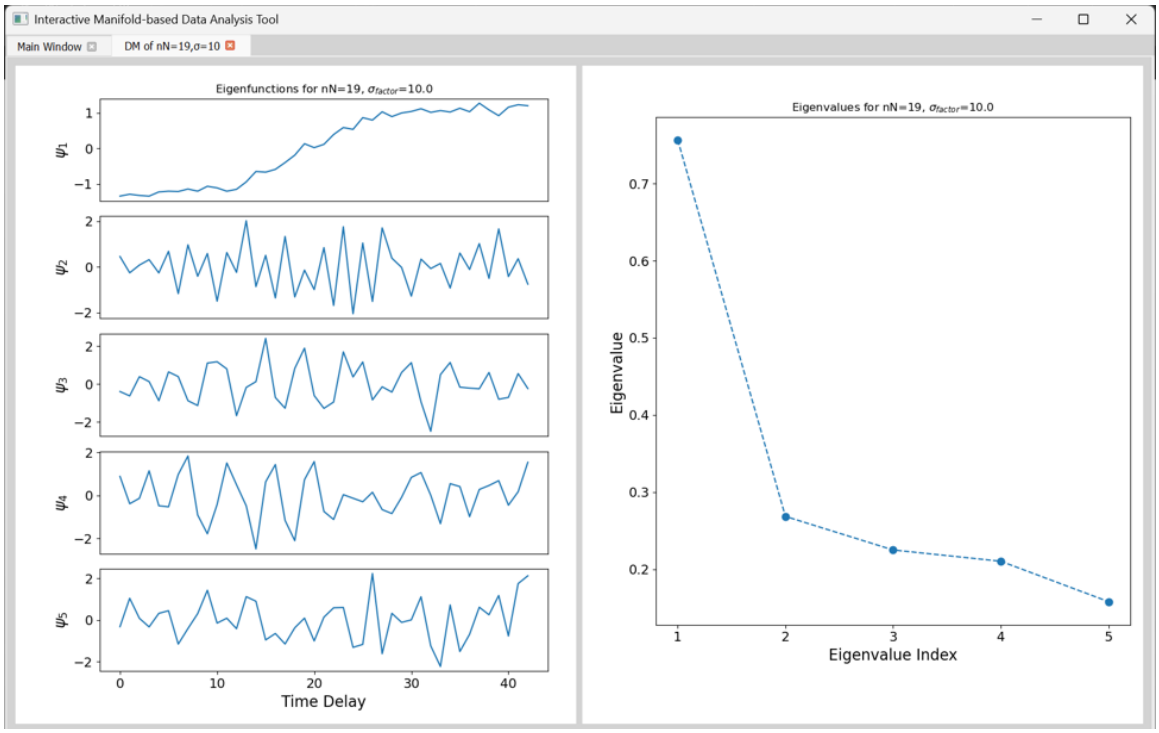

Figure 3. The second tab of the GUI shows eigenfunctions and eigenvalues from the diffusion map.

13. Run NLSA :

This button is for running the NLSA on the given data file for the selected nN and sigma values. Analysis progress shown in the progress bar #16. After the analysis, it plots chronos in a new tab as shown in Figure 4.

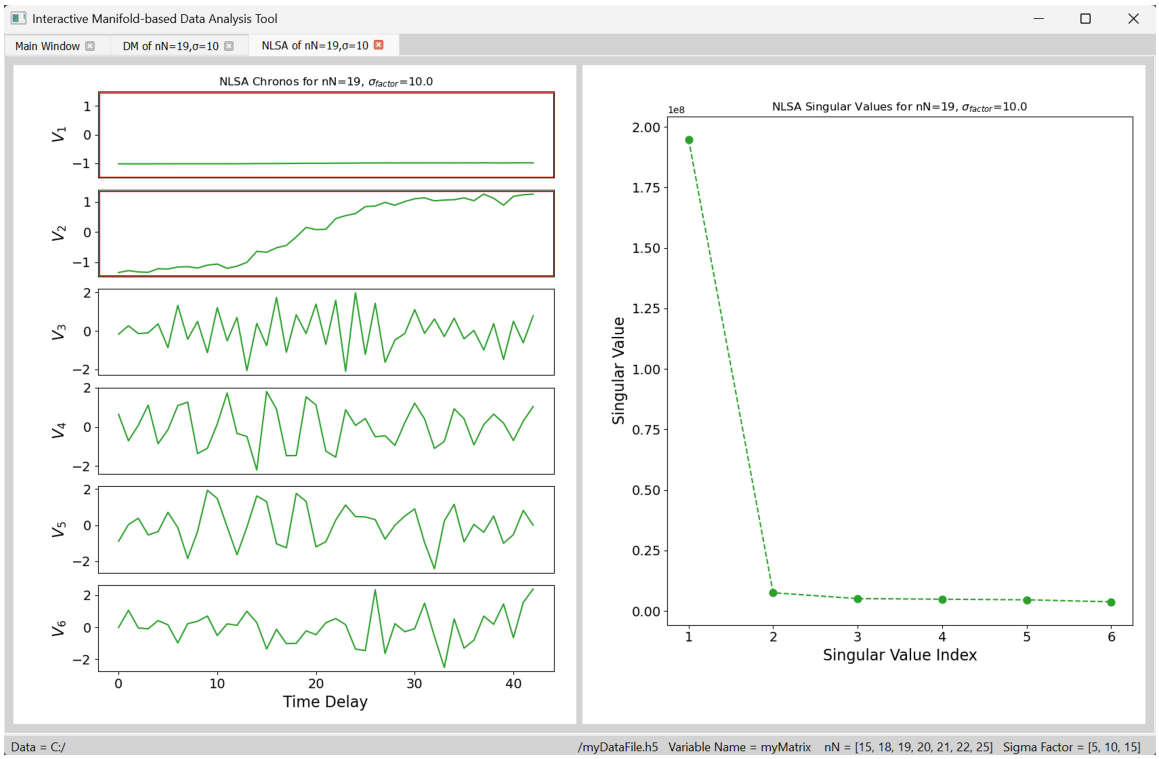

Figure 4. The new tab of the GUI showing NLSA chronos and singular values. The chrono plots are interactive. Therefore, the desired chronos can be selected for the NLSA reconstruction. The selected chronos are highlighted in red colour, and the corresponding information is shown in the main tab.

14. NLSA reconstruction:

This is a button used to perform NLSA reconstructiion using the selected modes (chronos). The results are saved in a **.h5* file in the current working directory.

15. Data Info:

The data information related to the selected data file (button #1), and the suggested values for running the parameter search analysis are shown here.

16. Progress Bar:

The progress bar reflects the stage of the analysis and helps users to estimate the running time of the code for their data.

GUI Runtime files and Result files

All the GUI files are stored in a folder named *temp_data_{timestamp}*. Every time one runs the GUI, the code will create a new folder for storing the generated files, such as *heatmap.png*, *fusion.png*, etc. Inside this folder, we create one more file for storing the run data for future use.